Top-Down and Bottom-Up Integration Testing

Introduction to Integration Testing

Integration testing is a level of software testing in which two or more individually tested modules are combined and tested together to verify that they interact correctly.

In software development, a large application is divided into smaller modules or components. Each module may work correctly on its own, but problems can occur when modules communicate with one another.

For example, in an online shopping application, the following modules may be developed separately:

- Login module
- Product catalogue module
- Shopping cart module
- Payment module
- Order management module

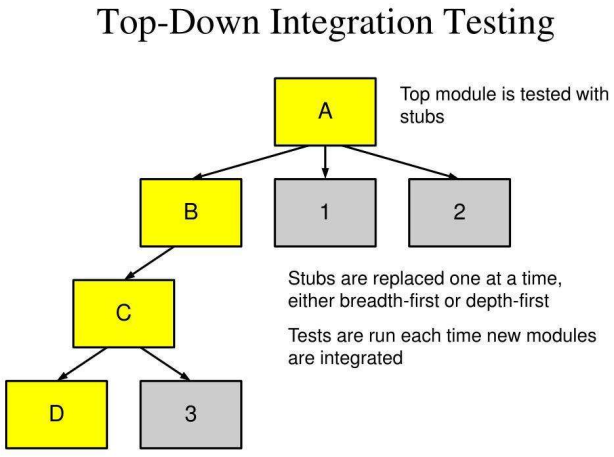
Integration testing checks whether these modules exchange data correctly and work together as expected.

Objectives of integration testing

6. To detect errors in communication between modules.
7. To verify the correctness of data transfer between components.
8. To check whether interfaces work according to specifications.
9. To identify errors in function calls and parameter passing.
10. To verify the interaction between dependent modules.
11. To detect integration defects before system testing.

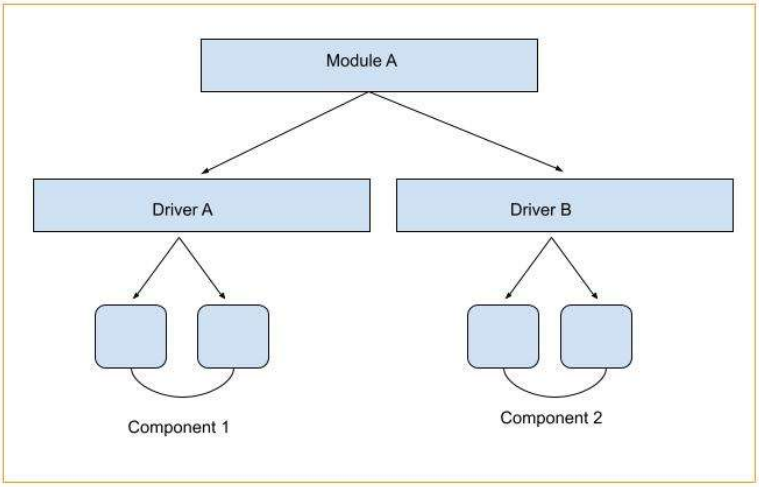
Types of integration testing

The two major approaches are:



1. Top-Down Integration Testing

Testing starts with the top-level or main control module and gradually integrates lower-level modules. Stubs are used when lower-level modules are not yet available.



2. Bottom-Up Integration Testing
Testing starts with the lowest-level modules and gradually integrates higher-level modules. Drivers are used when higher-level modules are not yet available.

Top-Down Integration Testing

Definition

Top-down integration testing is an integration testing technique in which testing begins with the highest-level module and proceeds downward through the module hierarchy until all lower-level modules are integrated and tested.

The main control module is tested first, and lower-level modules are gradually connected to it.
If a lower-level module is unavailable, a temporary program called a **stub** is used to simulate its behaviour.

Diagram of top-down integration testing
A — Main Module
B — Module
C — Module
D — Stub
E — Stub

Testing proceeds from the main module toward lower-level modules. Stubs temporarily replace modules that have not been developed or integrated.

How top-down integration testing works

- **Identify the main module:** Select the top-level module that controls the overall application.
- **Test the main module:** Check its basic control logic and decision-making.
- **Select lower-level modules:** Identify the modules directly called by the main module.
- **Use stubs where necessary:** Replace unavailable lower-level modules with temporary implementations.
- **Integrate modules gradually:** Replace stubs with actual modules as they become available.
- **Perform regression testing:** Retest previously working functionality after every integration.
- **Continue downward:** Repeat the process until all modules have been integrated and tested.

Example

Consider a college management system with this module hierarchy:

12. A: Main system
13. B: Student registration
14. C: Fee management
15. D: Student database
16. E: Payment processing

The hierarchy is:

A: Main System

B: Registration

C: Fee Management

D: Database

E: Payment

In top-down testing:

17. First, test the Main System.
18. Next, integrate and test Student Registration and Fee Management.
19. If the database and payment modules are unavailable, use stubs for them.
20. Replace the stubs with the real modules when available.
21. Test the complete integrated system.

What is a stub?

A **stub** is a temporary, simplified program that replaces a lower-level module that has not yet been implemented or integrated. A stub returns predefined responses so that the calling module can be tested.

Example: Suppose the registration module calls a database module to retrieve student details, but the database module is not ready.

A stub might return: python

```
def get_student_details(student_id):
   return {
      "id": student_id,
      "name": "Rahul",
      "status": "Registered"
   }
```

This allows the registration module to be tested without the actual database.

Key point: Stubs simulate called modules and are commonly used in top-down integration testing.

Advantages of top-down integration testing

1. **Early testing of control logic:** The main module and major control flow are tested early.
2. **Early prototype availability:** A basic working version of the application can be demonstrated early.
3. **Early detection of architectural errors:** Problems in the main control structure may be discovered sooner.
4. **No drivers required:** Since testing starts from the top, separate drivers for higher-level modules are generally unnecessary.
5. **Early interface testing:** Interfaces between high-level and lower-level modules can be examined as integration progresses.

Disadvantages of top-down integration testing

1. **Many stubs may be required:** Unavailable lower-level modules must be simulated.
2. **Low-level modules are tested late:** Errors in database access, calculations, or utility functions may be discovered later.
3. **Stub development takes time:** Stubs must behave realistically enough to support meaningful tests.
4. **Limited realistic testing:** Simplified stubs may not reproduce the behaviour of actual modules.
5. **Detailed functionality may receive less early attention:** Testing initially focuses on high-level control flow rather than the full implementation of lower-level components.

When is top-down testing suitable?

Top-down testing is particularly useful when:

- The main control module is available early.
- Overall application flow is important.
- High-level design and architecture need early validation.
- A working prototype is needed quickly.
- Lower-level modules are still under development.

Bottom-Up Integration Testing

Definition

Bottom-up integration testing is an integration testing technique in which testing begins with the lowest-level modules and gradually proceeds upward toward higher-level modules.

Low-level modules are combined into groups, tested, and then integrated with modules at higher levels. When a higher-level calling module is unavailable, a temporary program called a **driver** is used to invoke the lower-level modules.

Diagram of bottom-up integration testing

D — Module
E — Module
B — Module
C — Module
A — Main Module

Testing starts at the bottom and moves upward. Drivers temporarily invoke lower-level modules until the real higher-level modules are available.

How bottom-up integration testing works

1. **Identify the lowest-level modules:** Find the modules that do not depend on other application modules.
2. **Test individual modules:** Check their functionality and outputs.
3. **Create clusters:** Group related low-level modules that work together.
4. **Use drivers:** Create temporary calling programs to supply inputs and invoke the modules under test.
5. **Integrate the clusters:** Combine tested low-level modules with their parent modules.
6. **Perform regression testing:** Ensure that integration has not introduced new errors.
7. **Move upward:** Continue integrating and testing higher-level modules until the main module is included.

Example

Consider the same college management system:

- A: Main system
- B: Student registration

- C: Fee management
- D: Student database
- E: Payment processing

In bottom-up testing:

- First, test the Student Database and Payment Processing modules.
- Use drivers to invoke them because their parent modules may not yet be available.
- Integrate the database with Student Registration.
- Integrate Payment Processing with Fee Management.
- Integrate the resulting groups with the Main System.
- Test the complete application.

What is a driver?

A **driver** is a temporary program that calls or invokes a lower-level module when the actual higher-level calling module is unavailable. A driver supplies test inputs, calls the module, and helps examine its output.

Example: python

```
def calculate_fee(amount, discount):
    return amount - discount

# Temporary driver
def test_fee_module():
    result = calculate_fee(1000, 100)
    print("Expected: 900")
    print("Actual:", result)

test_fee_module()
```

The temporary test function acts as a simple driver by calling the fee calculation module and displaying its result.

Key point: Drivers simulate calling modules and are commonly used in bottom-up integration testing.

Advantages of bottom-up integration testing

1. **Early testing of low-level functionality:** Utility functions, calculations, database operations, and data-processing modules can be tested early.
2. **Fewer stubs are needed:** Lower-level modules are real implementations rather than simulated dependencies.
3. **Fault localization is easier:** Problems in low-level modules and their interfaces can often be isolated more easily.
4. **Useful for reusable components:** Shared utilities and libraries can be thoroughly tested before being integrated into larger systems.
5. **Detailed functionality is tested early:** The fundamental operations of the application receive attention at the beginning.

Disadvantages of bottom-up integration testing

1.Drivers may be required: Temporary calling programs must be created for modules whose parents are unavailable.

2.Main control logic is tested late: The complete application flow may not be available until later.

3.No early complete prototype: High-level functionality may be difficult to demonstrate in the initial stages.

4.High-level design errors may be detected later: Problems involving overall coordination and control flow may remain hidden until upper modules are integrated.

5.Complex interactions may be delayed: Some system-wide behaviour cannot be evaluated until enough modules have been combined.

When is bottom-up testing suitable?

Bottom-up testing is especially useful when:

- Low-level modules are ready before high-level modules.
- The application contains complex calculations or data-processing components.
- Database and utility modules are critical.
- Reusable components require early validation.
- Developers want to test fundamental functionality before the complete system is assembled.

Difference Between Top-Down and Bottom-Up Integration Testing

Basis	**Top-Down Testing**	**Bottom-Up Testing**
Starting point	Top-level module	Lowest-level modules
Direction	Top to bottom	Bottom to top
First priority	Main control logic	Fundamental operations
Temporary component	Stubs	Drivers
Lower-level testing	Usually later	Usually earlier
High-level testing	Usually earlier	Usually later
Main control flow	Evaluated early	Evaluated later
Low-level functionality	Evaluated later	Evaluated early
Prototype	Can be available early	Usually available later
Main benefit	Early validation of architecture and control flow	Early validation of low-level functionality
Main limitation	Dependence on stubs	Dependence on drivers
Suitable for	Systems where high-level control is critical	Systems where low-level components are critical

Difference Between Stub and Driver

Basis	**Stub**	**Driver**
Purpose	Simulates a called module	Simulates a calling module
Used in	Top-down integration	Bottom-up integration
Position	Usually below the module under test	Usually above the module under test
Role	Provides responses to calls	Sends inputs and calls the module
Replaces	An unavailable lower-level module	An unavailable higher-level module
Main function	Responds to the caller	Invokes the component under test

Hybrid or Sandwich Integration Testing

Hybrid integration testing, also called **sandwich integration testing**, combines top-down and bottom-up approaches.

Testing proceeds simultaneously from the top and the bottom toward the middle layers.

- The top-down approach tests high-level modules using stubs.
- The bottom-up approach tests low-level modules using drivers.
- The middle layer is integrated when the two testing paths meet.

Top-level modules

Middle-level modules

Low-level modules

Testing progresses downward from the top and upward from the bottom, meeting in the middle.

Advantages

- High-level and low-level modules can be tested in parallel.
- It can reduce the time needed to integrate a large application.
- Both architectural control flow and detailed functionality receive early attention.

Disadvantages

- Test planning and coordination can be more complex.
- Both stubs and drivers may be required.
- It may be unsuitable for small applications where parallel integration provides little benefit.

Bi-Directional Integration Testing

Definition

Bi-directional integration testing is an integration testing approach that combines the principles of top-down and bottom-up integration testing. Testing proceeds from the top-level modules downward and from the bottom-level modules upward, with both approaches meeting at the intermediate modules. It is commonly associated with sandwich or hybrid integration testing.

The purpose is to test high-level control logic and low-level functionality in parallel, reducing the time required to integrate a large software system.

2. Diagram of Bi-Directional Integration Testing

A — Main Module

B — Intermediate Modules

Top-down

Bottom-up

C — Low-Level Modules

Testing progresses downward from the main module and upward from the low-level modules until the integration paths meet.

How Bi-Directional Integration Testing Works

1. **Divide the system into modules:** Identify the high-level, intermediate, and low-level modules.
2. **Start top-down testing:** Begin with the main module and integrate the modules below it. Use stubs for unavailable lower-level modules.

3. **Start bottom-up testing:** Begin with the lowest-level modules and integrate upward. Use drivers for unavailable higher-level modules.
4. **Test both paths in parallel:** Verify high-level control flow and low-level functionality independently.
5. **Integrate intermediate modules:** Connect the two testing paths at the intermediate level.
6. **Perform regression testing:** Rerun relevant tests to ensure integration has not broken existing functionality.
7. **Test the complete system:** Verify that all modules work together correctly.

Example

Consider an online shopping application with the following modules:

- A: Main application
- B: Product management
- C: Order management
- D: Database
- E: Payment processing

Testing procedure

Top-down direction:

- Test the Main Application.
- Integrate Product Management and Order Management.
- Use stubs if the Database or Payment Processing modules are unavailable.

Bottom-up direction:

- Test the Database and Payment Processing modules.
- Use drivers to call and test them.
- Integrate them with their respective higher-level modules.

Final integration:

- Connect the tested modules.
- Verify product selection, order placement, payment processing, and database updates.
- Test the complete shopping workflow.

Advantages

Saves time: High-level and low-level modules can be tested in parallel.

Early detection of errors: Problems in control flow and low-level functionality can be discovered early.

Improved test coverage: Both architectural behaviour and detailed operations receive attention.

Suitable for large systems: It is useful when an application contains several layers of modules.

Efficient integration: The two testing paths can reduce delays while waiting for all modules to become available.

Disadvantages

Complex planning: Two testing paths must be coordinated.

Both stubs and drivers may be required: This increases test-development effort.

Integration can be complicated: The top-down and bottom-up paths must meet correctly.

Additional resources may be needed: Parallel testing may require more testers or development resources.

Not always suitable for small systems: The extra setup may provide little benefit for simple applications.

Difference Between Top-Down, Bottom-Up, and Bi-Directional Testing

Basis	Top-Down	Bottom-Up	Bi-Directional
Starting point	Top-level modules	Low-level modules	Both ends
Direction	Downward	Upward	Both directions
Test components	Mainly stubs	Mainly drivers	Both stubs and drivers may be used
Early focus	Control logic	Basic functionality	Both
Testing style	One direction	One direction	Parallel paths
Best suited for	High-level control validation	Low-level component validation	Large, layered systems

Types of Software Testing

- Software Testing is a critical phase of the Software Development Life Cycle (SDLC) that ensures a software application works correctly and meets user requirements. It helps identify defects and improve the overall quality, reliability, and performance of the software.
  - Ensures the application functions as expected and meets user needs.
  - Helps detect bugs early and improves software quality.
  - Enhances reliability, security, and performance of the system.

How Software Testing Is Classified
Software testing can be classified into different categories based on how testing is performed, the testing technique used, the level at which the software is tested, and the purpose of testing.

- **Based on Execution Method:** Manual Testing and Automation Testing.
- **Based on Testing Technique:** Black Box Testing, White Box Testing, and Gray Box Testing.
- **Based on Testing Level:** Unit Testing, Integration Testing, System Testing, and Acceptance Testing.
- **Based on Testing Purpose:** Functional Testing and Non-Functional Testing.

Types Based on Execution Method
Software testing can be classified based on how test cases are executed, either manually by testers or automatically using testing tools and scripts.

a. **Manual Testing** Manual Testing is a software testing technique in which testers execute test cases manually without using automation scripts or tools. Helps identify UI/UX issues related to layout, design, and text. Does not require developing automation scripts or frameworks. Easily adapts to frequent or unexpected changes in the application.

b. **Automation Testing** Automation Testing is a testing technique in which test cases are executed using scripts and automation tools. It is mainly used for repetitive and time-consuming tests to improve efficiency, consistency, and test coverage. Runs tests

automatically, saving time and effort. Enables the execution of a large number of test cases efficiently. Minimizes the effort required for repetitive testing tasks.

Types Based on Testing Technique

Software testing can be classified based on the technique used to examine an application. Depending on the tester's knowledge of the internal implementation, testing techniques are broadly categorized into Black Box Testing, White Box Testing, and Gray Box Testing.

a. **White Box Testing** White Box Testing is a software testing technique in which the internal structure, logic, and source code of an application are examined. It is typically performed by developers to verify that the code functions correctly and efficiently. Focuses on internal code, logic, and program structure. Requires knowledge of programming and code implementation. Helps identify errors in loops, conditions, paths, and data flow.
b. **Black Box Testing** Black-Box Testing is a software testing technique in which the functionality of an application is tested without knowledge of its internal code or structure. It focuses on inputs and expected outputs to verify whether the system behaves correctly. Focuses on the application's external functionality. Does not require knowledge of the internal code or implementation. Validates the application by comparing actual results with expected results.
c. **Gray Box Testing** Gray Box Testing is a software testing technique that combines aspects of Black Box and White Box Testing. The tester has partial knowledge of the internal structure while testing the application from an external perspective. Combines functional and structural testing approaches. Requires partial knowledge of the application's internal implementation. Helps identify defects related to data flow and integration.

Types Based on Testing Level

Software testing can be classified based on the level at which different components of an application are tested. Testing levels generally progress from individual components to the complete system to ensure that the software works correctly at each stage.

a. **Unit Testing** Unit Testing verifies individual units or components, such as functions, methods, or classes, in isolation. Tests individual components independently. Helps identify defects early in development. Usually performed by developers.
b. **Integration Testing** Integration Testing verifies whether different modules or components work correctly together. Tests interactions between integrated components. Verifies data flow and communication between modules. Helps identify interface and integration defects.
c. **System Testing** System Testing evaluates the complete and integrated application as a whole to verify that it meets specified requirements. Tests the complete system. Verifies functional and non-functional requirements. Identifies defects in the overall system behavior.

d. **Acceptance Testing** Acceptance Testing verifies whether the software meets business and user requirements before release or deployment. Validates the software against business requirements. Often performed by users, clients, or business stakeholders. Determines whether the software is ready for release.

Types Based on Testing Purpose
Software testing can be classified based on the purpose of testing, such as validating functionality, performance, security, reliability, and other quality attributes.

a. **Functional Testing** Functional Testing verifies whether the software functions according to the specified business requirements.

Types of Functional Testing
- Smoke Testing: Checks the basic and critical functionalities of a new build to determine whether it is stable enough for further testing.
- Sanity Testing: Performs a focused validation of specific changes or bug fixes to ensure they work as expected.
- Regression Testing: Verifies that recent changes or bug fixes have not negatively affected existing functionality.
- Retesting: Verifies whether a previously reported defect has been successfully fixed.
- API Testing: Verifies API functionality by validating requests, responses, data exchange, and business logic.
- Interface Testing: Verifies whether different systems or components communicate and exchange data correctly.

b. **Non-Functional Testing**

Non-Functional Testing evaluates how well the software performs under different conditions. It focuses on quality attributes such as performance, security, usability, reliability, and scalability.
Types of Non-Functional Testing
- Performance Testing: Evaluates the speed, responsiveness, and stability of an application under different workloads.
- Security Testing: Identifies vulnerabilities and weaknesses that could compromise the application's security.
- Usability Testing: Evaluates how easy and intuitive the application is for users.
- Reliability Testing: Verifies whether the software performs consistently over a specified period.
- Compatibility Testing: Verifies that the application works correctly across different browsers, devices, operating systems, and environments.

Types of Performance Testing
- Load Testing: Evaluates the application's performance under expected user load and normal working conditions. It helps identify performance bottlenecks before deployment.
- Stress Testing: Tests the system beyond its normal operating limits to determine its breaking point. It helps assess stability and error-handling capabilities under extreme conditions.
- Spike Testing: Measures how the application responds to sudden increases or decreases in user load. It ensures the system can handle unexpected traffic spikes effectively.
- Endurance (Soak) Testing: Checks the application's performance over an extended period under a sustained workload. It helps identify memory leaks and resource utilization issues.
- Volume Testing: Evaluates the system's ability to handle large volumes of data. It ensures that performance remains acceptable as the amount of data increases.

Types of Integration Testing

- Big Bang Integration Testing: All modules are integrated simultaneously and tested as a complete system. While simple to implement, identifying the source of defects can be challenging.
- Top-Down Integration Testing: Testing begins with the top-level modules and progressively integrates lower-level modules. Stub programs are used to simulate modules that are not yet developed.
- Bottom-Up Integration Testing: Testing starts with the lowest-level modules and gradually moves upward. Driver programs are used to simulate higher-level modules during testing.
- Sandwich (Hybrid) Integration Testing: This approach combines both Top-Down and Bottom-Up testing methods. It can allow modules at different levels to be tested simultaneously.

Other Types of Testing

In addition to testing classifications based on execution method, technique, level, and purpose, several specialized testing types are used to evaluate specific aspects of software.

- Alpha Testing: Testing performed internally in a controlled environment before the software is released to external users.
- Beta Testing: Testing performed by selected real users in a real-world environment before the final release.
- Exploratory Testing: An unscripted approach in which testers explore the application and design tests based on their observations.
- Ad Hoc Testing: Informal testing performed without predefined test cases to identify unexpected defects.
- Installation Testing: Verifies that software can be installed, upgraded, and uninstalled successfully.
- Globalization Testing: Verifies that an application supports multiple languages, regions, and cultural settings.
- Localization Testing: Ensures that an application is correctly adapted for a specific language, region, or culture.
- Graphical User Interface Testing: Validates the appearance and functionality of user interface elements such as buttons, menus, forms, and layouts.
- Accessibility Testing: Verifies whether an application can be effectively used by people with disabilities.
- End-to-End Testing: Tests complete application workflows from start to finish.

Advantages of Software Testing

Software testing provides several benefits by helping organizations identify defects, improve software quality, and ensure that applications meet user and business requirements.

- Ensures that the application works according to specified requirements.
- Helps identify and fix issues before they become more costly.
- Early defect detection can reduce the cost of fixing problems later in development.
- Improves the reliability and overall quality of the application.
- Provides users with a more reliable and better-quality application.

Limitations of Software Testing

Although software testing is essential for ensuring software quality, it has certain limitations and cannot guarantee that an application is completely defect-free.

- Testing can be a time-consuming process, especially for large and complex applications.
- It can increase development costs because testing requires resources, tools, and skilled testers.
- Testing cannot guarantee the complete absence of defects.
- Testing every possible input, condition, and user scenario is often impractical.
- Some defects may remain undiscovered until the software is used in real-world conditions.

System Integration Testing

Introduction

System Integration is the process of combining different software systems, subsystems, modules, or external applications so that they function together as one complete system.
System Integration Testing (SIT) is a testing technique used to verify that these integrated systems communicate correctly, exchange data accurately, and work together according to the specified requirements.

For example, an online shopping application may connect:

- User management system
- Product management system
- Payment gateway
- Inventory management system
- Delivery and shipping system
- Notification service

System integration testing verifies that all these systems interact correctly.

Definition of System Integration Testing

System Integration Testing (SIT) is a type of software testing performed to verify the interfaces, communication, data flow, and interactions between integrated systems or subsystems.

Its primary purpose is to detect defects that occur when independently developed systems are connected.

Diagram of System Integration

User Interface
Application System
Payment System
Database
External Services
System integration connects software components and external services so they can exchange information and complete business processes.

Objectives of System Integration Testing

The main objectives are:

1. **Verify communication:** Ensure that different systems communicate correctly.
2. **Validate data exchange:** Check that data sent from one system is received and interpreted correctly by another.
3. **Test interfaces:** Verify APIs, web services, database connections, and other interfaces.
4. **Detect integration defects:** Identify errors caused by connecting independently developed systems.
5. **Verify end-to-end workflows:** Ensure that complete business processes work across multiple systems.
6. **Check error handling:** Confirm that failures, invalid responses, and timeouts are handled correctly.
7. **Verify security:** Check authentication, authorization, and secure data exchange between systems.
8. **Check compatibility:** Ensure that integrated systems work together with their respective protocols, formats, and configurations.

Example of System Integration Testing

Consider an online shopping system.

When a customer purchases a product, the following process occurs:

1. The customer selects a product.

2. The application checks product availability in the inventory system.
3. The customer makes payment through a payment gateway.
4. The order management system records the order.
5. The inventory system updates the available stock.
6. The notification service sends an order confirmation.

System integration testing checks whether each system communicates correctly with the others.

For example:

- Does the payment gateway return the correct payment status?
- Is the order created only when the payment rules are satisfied?
- Is the inventory updated correctly?
- Is the customer notified with the correct order details?
- What happens if the payment gateway times out?

These tests help identify problems that might not appear when each system is tested independently.

Types of System Integration Testing

1 Big-Bang Integration Testing

In this approach, multiple modules or systems are integrated together at once and then tested.

Advantages:

- Simple to organize when all components are ready.
- Less need for repeated integration steps.

Disadvantages:

- Defects are difficult to isolate.
- Testing starts late.
- Failures in one component can affect many other components.

2 Incremental Integration Testing

In this approach, systems are integrated and tested gradually, one component or group at a time.

Advantages:

- Defects are easier to identify and isolate.
- Testing can begin before every component is ready.
- Integration problems can be detected earlier.

Disadvantages:

- Requires more planning.
- May require temporary stubs, drivers, mocks, or service simulators.

Incremental integration includes top-down, bottom-up, and hybrid approaches.

3 API Integration Testing

API integration testing verifies communication between applications through application programming interfaces.

It checks:

- Request and response formats
- HTTP status codes
- Authentication
- Data validation
- Error responses
- API response times, where relevant

4 Third-Party Integration Testing

This verifies connections with external services such as payment gateways, email services, shipping providers, or identity providers.

It checks whether the application handles successful responses, failures, timeouts, and unexpected responses correctly.

System Integration Testing Process

1. **Requirement analysis**

Identify the systems, interfaces, data flows, and expected outcomes.

2. **Integration planning**

Choose the integration sequence, testing approach, tools, and test environment.

3. **Test case design**

Create test cases for valid data, invalid data, failures, and end-to-end workflows.

4. **Environment setup**

Configure APIs, databases, credentials, network connections, and test data.

5. **Execute integration tests**

Run tests across the connected systems and record actual results.

6. **Defect reporting and fixing**

Investigate interface and communication failures, correct defects, and retest.

8. **Regression testing**

Verify that fixes and changes have not broken existing integrations.

9. **Test completion**

Review coverage, unresolved defects, and results against exit criteria.

Important Test Cases

Test case	**Expected result**
Valid request between systems	Correct response is returned
Invalid input data	Appropriate validation error is returned
Payment succeeds	Payment and order status are consistent
Payment fails	Failure is handled according to business rules
API timeout	Timeout is handled without incorrect duplicate processing
Database connection fails	Error is reported and handled safely
Duplicate request is sent	Duplicate processing is prevented where required
Unauthorized request	Access is rejected
Incorrect response format	The receiving system handles the error appropriately
Inventory is updated	Stock information remains consistent

Advantages of System Integration Testing

1. Detects defects between connected systems.
2. Verifies correct data transfer and consistency.
3. Improves reliability of end-to-end business processes.
4. Helps identify interface and compatibility problems.
5. Validates interactions with third-party services.

6. Reduces the risk of failures after deployment.
7. Helps verify error handling across system boundaries.

Disadvantages of System Integration Testing

1. Requires a properly configured integration environment.
2. Can be difficult when many systems and interfaces are involved.
3. External services may be unavailable or unreliable during testing.
4. Failures can be difficult to reproduce across distributed systems.
5. Test data and credentials must be managed carefully.
6. Integration testing may require additional tools, time, and resources.

Difference Between Integration Testing and System Integration Testing

Basis	**Integration Testing**	**System Integration Testing**
Main focus	Interaction between modules or components	Interaction between systems or subsystems
Scope	Often within one application	Often across multiple applications or services
Example	Testing a login module with a user database module	Testing an application with an external identity provider
Interfaces	Module interfaces and function calls	APIs, network protocols, data formats, and service interfaces
Dependencies	Often internal components	Often internal and external systems
Typical concern	Module communication	Cross-system workflows and data consistency

Difference Between System Testing and System Integration Testing

Basis	**System Integration Testing (SIT)**	**System Testing**
Purpose	Verify interactions between connected systems	Verify the complete system against requirements
Main focus	Interfaces, communication, and data exchange	Overall functionality and non-functional requirements
Scope	Connected systems or subsystems	Complete integrated application
Example	Verify that a payment gateway updates an order correctly	Verify that a customer can complete the full purchase process and that the application meets its requirements
Typical timing	After relevant components are available for integration	After integration has produced a testable complete system

Scenario Testing and Defect Bash

Scenario Testing

Definition

Scenario testing is a software testing technique in which test cases are designed around realistic situations or user activities to verify how a system behaves in practical use.

A scenario describes a sequence of actions that a user or another system performs to achieve a particular goal.
Example: In an online shopping application, a customer logs in, searches for a product, adds it to the cart, makes payment, and receives an order confirmation. Testing this complete sequence is scenario testing.

Objectives of scenario testing

1. To verify real-world usage of software.
2. To test complete business workflows.
3. To identify defects that may not appear in isolated test cases.
4. To validate interactions between different modules.
5. To verify that the application meets user requirements.
6. To test both successful and unsuccessful user journeys.

Characteristics of scenario testing

- Based on realistic user situations.
- Focuses on complete workflows rather than only individual functions.
- Can involve multiple modules and external systems.
- Includes positive and negative scenarios.
- Helps identify usability, integration, and functional defects.

Steps involved in scenario testing

1. Understand requirements
Identify the purpose of the application and user expectations.

2. Identify user roles
Determine who will use the system, such as a customer, administrator, or employee.

3. Prepare scenarios
Write realistic situations based on business workflows.

4. Define test data
Prepare valid, invalid, and boundary data required for each scenario.

5. Execute scenarios
Perform the actions in the specified sequence.

6. Compare results
Compare actual behaviour with expected behaviour.

7. Report defects
Record unexpected results, steps to reproduce, and supporting evidence.

Example of scenario testing: Online shopping application

Scenario: A customer purchases a product successfully.

Step	Action	Expected result
1	Open the application	Home page is displayed
2	Log in with valid credentials	Customer logs in successfully

3	Search for a product	Relevant products are displayed
4	Add a product to the cart	Product appears in the cart
5	Enter the delivery address	Address is accepted
6	Complete payment	Payment is processed successfully
7	Confirm the order	Order number is generated
8	Check confirmation	Confirmation message is received

If the payment succeeds but the order is not created, the test reveals a potential integration or workflow defect.

Types of scenarios

- **Positive scenario:** Tests whether the system works correctly with valid inputs.
- **Negative scenario:** Tests how the system handles invalid inputs or failed operations.
- **End-to-end scenario:** Tests a complete workflow across multiple modules.
- **Alternative scenario:** Tests another valid route to achieve the same goal.
- **Exception scenario:** Tests unexpected situations, such as a network failure or payment timeout.

Advantages

1. Tests software in realistic situations.
2. Verifies complete business processes.
3. Helps identify defects across module boundaries.
4. Improves confidence in the application's usability.
5. Helps validate business requirements.

Disadvantages

1. Complex scenarios can take a long time to execute.
2. Many test data combinations may be required.
3. Failures involving several modules can be difficult to diagnose.
4. Some scenarios may be difficult to automate.
5. Exhaustively testing every possible user journey is usually impractical.

Defect Bash

Definition

A defect bash is a focused testing session in which a group of people systematically explores a software application to identify and report as many valid defects as possible within a defined period.

Participants may include testers, developers, business analysts, product owners, designers, and other stakeholders.

A defect bash is often conducted before a release or during a testing milestone to discover defects that routine test cases may have missed.
Simple meaning: A defect bash is a collaborative activity in which a team tests software together to find defects quickly.

Objectives of a defect bash

1. To discover defects before software release.
2. To encourage exploratory testing.
3. To identify defects missed by existing test cases.
4. To obtain feedback from different perspectives.

5. To test usability and real-world workflows.
6. To improve the overall quality of the application.

How a defect bash works

1. **Planning** Define the application area, testing goals, participants, duration, and reporting method.
2. **Environment setup** Provide access to the test build, test accounts, devices, and test data.
3. **Kick-off** Explain the rules, scope, known issues, and how defects should be recorded.
4. **Exploratory testing** Participants explore features, follow scenarios, and try unusual or unexpected inputs.
5. **Defect reporting** Record each issue with steps to reproduce, expected and actual results, and evidence.
6. **Triage** Review reports, remove duplicates, confirm valid defects, and assign severity and priority.
7. **Follow-up** Fix defects, retest corrections, and perform regression testing where needed.

Example of a defect bash

Suppose a team is testing a banking application before release.

Participants test:

- Login and logout
- Money transfers
- Balance display
- Transaction history
- Password reset
- Mobile responsiveness
- Network interruption during transactions

During the session, a tester discovers that the application displays a successful transfer message even though the transaction has failed.

The tester records the defect with the following details:

Field	**Example**
Defect title	Incorrect success message after failed transfer
Module	Money transfer
Steps	Log in, initiate a transfer, simulate a failed transaction
Expected result	A failure message is displayed and the transfer status remains accurate
Actual result	The application displays a success message
Severity	Potentially high, depending on transaction impact
Evidence	Screenshot, logs, or transaction reference from the test environment

The team then investigates, assigns the defect, fixes it, and retests the transaction flow.

Types of defects commonly found

- Functional defects
- User interface defects

- Usability problems
- Integration defects
- Validation errors
- Compatibility issues
- Performance problems noticeable during use
- Security-related issues discovered through authorized testing

Advantages of a defect bash

1. **Collaborative testing:** Different participants identify different types of problems.
2. **Rapid feedback:** Many features can be explored within a short period.
3. **Fresh perspectives:** Participants may discover issues missed by the original development team.
4. **Good for usability testing:** Realistic exploration reveals confusing workflows.
5. **Improved release confidence:** Important defects can be discovered before deployment.

Disadvantages of a defect bash

1. **Unpredictable coverage:** Participants may overlook some features.
2. **Duplicate reports:** Several people may discover the same defect.
3. **Variable report quality:** Some findings may lack sufficient reproduction details.
4. **Resource requirements:** Multiple people must be available at the same time.
5. **Not a replacement for planned testing:** A defect bash cannot guarantee complete coverage or replace regression and requirement-based testing.

Difference Between Scenario Testing and Defect Bash

Basis	**Scenario Testing**	**Defect Bash**
Meaning	Testing based on realistic situations and workflows	Collaborative session to discover defects
Main objective	Verify expected behaviour in practical use	Find as many valid defects as possible
Approach	Usually planned around defined scenarios	Often exploratory and time-boxed
Participants	Commonly testers, though others may participate	Multiple roles may participate
Test cases	Defined scenarios guide execution	Test charters or exploratory ideas may guide execution
Focus	Correctness of user journeys	Discovery of unexpected issues
Documentation	Scenario steps and results	Defect reports and evidence
Timing	Throughout the testing lifecycle	Often at a milestone or before release
Outcome	Verified workflow behaviour and identified failures	A collection of validated defect reports

Can Scenario Testing and Defect Bash Be Used Together?

Yes. They complement one another.

For example, during a defect bash for an e-commerce application:

1. The team uses scenario testing to check registration, login, product selection, payment, and order confirmation.

2. Participants then explore unusual cases, such as double-clicking the payment button or refreshing the page during checkout.
3. Any unexpected behaviour is recorded as a potential defect.
4. The team reviews the reports, fixes valid defects, and performs regression testing.

Scenario testing provides structured coverage, while a defect bash encourages broader exploration.

Functional vs Non-Functional Testing

Introduction

Software testing is the process of evaluating a software application to identify defects and verify that it meets specified requirements.

Software testing is broadly classified into two categories:

- **Functional testing:** Checks what the software does.
- **Non-functional testing:** Checks how well the software works.

For example, in an online shopping application:

- Functional testing checks whether a customer can log in, add products to the cart, and make payments.
- Non-functional testing checks whether the website loads quickly, handles thousands of users, protects customer data, and works across different devices.

Functional Testing

Definition

Functional testing is a type of software testing that verifies whether an application behaves according to its specified functional requirements.

It focuses on the features, operations, inputs, outputs, and business rules of the software.

Objectives of functional testing

1. Verify that every feature works correctly.
2. Check whether valid inputs produce expected outputs.
3. Verify the handling of invalid inputs.
4. Ensure that business rules are implemented correctly.
5. Identify functional defects.
6. Confirm that user requirements are satisfied.

Examples of functional testing

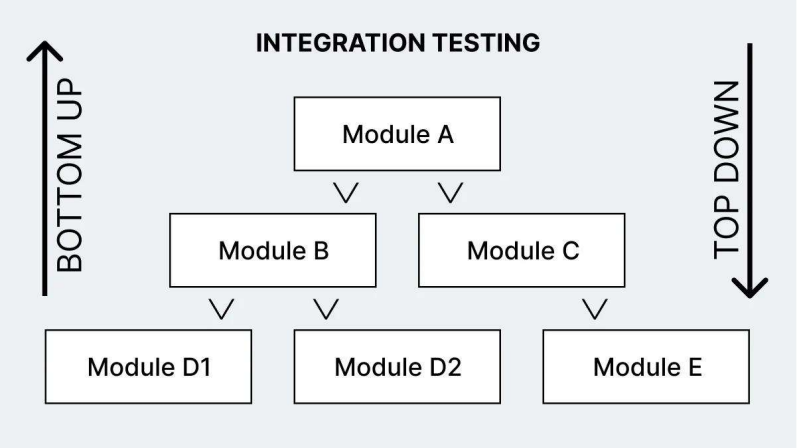
Consider a banking application.

- Verify that a user can log in with valid credentials.
- Verify that an incorrect password produces an error.
- Verify that a money transfer updates the account balances correctly.
- Verify that the transaction history displays the correct details.
- Verify that a user can reset a forgotten password.

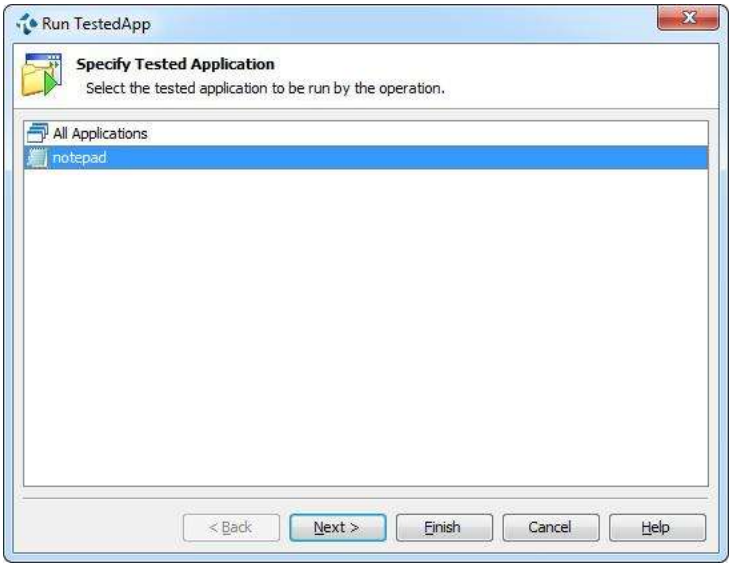
Types of functional testing

1. Unit Testing

Tests individual functions, methods, or components in isolation.



2. Integration Testing
Checks whether different modules or systems interact correctly.



3. System Testing
Tests the complete integrated application against its requirements.

4. User Acceptance Testing (UAT)
Checks whether the software meets business needs and is suitable for users.



5. Regression Testing
Checks whether changes or bug fixes have broken existing functionality.

Other common functional testing techniques include smoke testing, sanity testing, and interface testing.

Example of functional testing
Feature: Login page

Test input	Expected result
Valid username and password	Login succeeds
Invalid password	Appropriate error is displayed
Empty username	Required-field validation appears
Empty password	Required-field validation appears
Locked account	Access is denied according to the account rules

Advantages of functional testing
- Verifies business requirements.
- Helps detect errors in application features.
- Ensures that inputs and outputs behave correctly.
- Improves reliability of application workflows.
- Helps confirm that the software meets user expectations.

Limitations of functional testing
- Does not necessarily evaluate performance under heavy load.
- May not identify security weaknesses unless security checks are included.
- Does not automatically verify usability or accessibility.
- Can require many test cases to cover different inputs and business rules.

3. Non-Functional Testing

Definition

Non-functional testing is a type of software testing that evaluates the quality characteristics of a software system, such as performance, reliability, security, usability, scalability, and compatibility.
It focuses on **how well the system performs its functions**, rather than only whether those functions work.

Objectives of non-functional testing
19. Evaluate application performance.
20. Check system stability and reliability.
21. Verify security and data protection.
22. Evaluate usability and accessibility.
23. Check compatibility with browsers, operating systems, and devices.
24. Determine how the application behaves under different workloads.
25. Verify compliance with relevant quality requirements.

Types of non-functional testing
1. Performance Testing
Evaluates response time, throughput, resource usage, and overall system performance.

2. Load and Stress Testing
Load testing checks expected workloads. Stress testing evaluates behaviour beyond normal operating limits.
3. Security Testing
Evaluates vulnerabilities, access controls, authentication, and protection of information.
4. Compatibility Testing
Checks whether the application works across supported devices, browsers, operating systems, and environments.
5. Usability Testing
Evaluates how easily users can understand, navigate, and operate the application.
6. Reliability and Recovery Testing
Evaluates consistent operation over time and the ability to recover from failures.

Other non-functional testing activities include scalability testing, accessibility testing, installation testing, and maintainability evaluation.

Example of non-functional testing
Application: Online banking website

Test	**Expected result**
Performance testing	Pages respond within the specified time limit
Load testing	Application handles the expected number of simultaneous users
Stress testing	Application behaves predictably when pushed beyond normal capacity
Security testing	Unauthorized users cannot access protected account data
Compatibility testing	Application works on supported browsers and devices
Usability testing	Users can complete common banking tasks easily
Recovery testing	Application recovers appropriately after a simulated failure

Advantages of non-functional testing
- Improves application speed and responsiveness.
- Helps identify scalability limitations.
- Evaluates security and reliability.
- Improves user experience.
- Checks compatibility across environments.
- Reduces the risk of system failures under real-world conditions.

Limitations of non-functional testing
- May require specialized tools and expertise.
- Can be expensive and time-consuming.
- Often requires realistic infrastructure and workloads.
- Results may vary with hardware, network conditions, and environment.
- Some quality characteristics are difficult to measure precisely.

Difference Between Functional and Non-Functional Testing

Basis	**Functional Testing**	**Non-Functional Testing**
Meaning	Tests what the system does	Tests how well the system works
Main focus	Features and business logic	Quality characteristics

Based on	Functional requirements	Non-functional requirements
Main question	Does the feature work correctly?	Is the system fast, secure, reliable, and usable enough?
Examples	Login, registration, payment, calculations	Performance, security, usability, scalability
Inputs and outputs	Verifies correctness of results	Measures characteristics such as response time and stability
Test criteria	Expected behaviour and business rules	Quality targets and measurable thresholds
Tools	Unit test frameworks, UI automation tools, API clients	Load-testing, security-testing, monitoring, and usability tools
Defects found	Incorrect calculations, broken features, invalid workflows	Slow responses, vulnerabilities, crashes under load, usability issues
Typical measures	Pass/fail against expected results	Response time, throughput, availability, vulnerability findings, task success
Importance	Ensures required features work	Ensures the software is fit for real-world use

Example: Online Shopping Application

Suppose an online shopping website has a checkout feature.

Functional testing

- Verify that a product can be added to the cart.
- Verify that the discount code is applied correctly.
- Verify that the total price is calculated accurately.
- Verify that a successful payment creates an order.

Non-functional testing

- Verify that checkout loads within the specified response time.
- Verify that the site handles thousands of concurrent users if required.
- Verify that payment information is adequately protected.
- Verify that checkout works on supported mobile devices and browsers.

A website may pass functional testing but fail non-functional testing. For example, checkout might calculate the correct price but take 30 seconds to load under normal traffic.

Similarly, a website might be fast and secure but fail functional testing because it calculates discounts incorrectly.

Therefore, both types of testing are necessary.

Relationship Between Functional and Non-Functional Testing

Functional and non-functional testing complement one another. They are not mutually exclusive, and some testing activities may evaluate both.

For example, a performance test may verify that a payment API returns the correct response while also measuring how quickly it responds under load.

The distinction depends on the primary test objective:

- If the objective is correctness of behaviour, the focus is functional.
- If the objective is a quality characteristic such as speed or reliability, the focus is non-functiona

Software Testing and Verification

Design/Architecture Verification

Introduction

Software design and architecture verification is an important activity in the Software Development Life Cycle (SDLC). It checks whether the proposed software design and system architecture conform to specified requirements, design principles, standards, and constraints.
Verification is often described as **"Are we building the product right?"** It focuses on checking work products, such as requirements, architectural diagrams, design documents, interfaces, and component specifications, against their requirements.

Architecture verification usually takes place during the design phase and continues as the system evolves. Detecting architectural problems early is less expensive than correcting them after implementation or deployment.

Objectives of design and architecture verification

The major objectives are:

1. **Requirement compliance:** Ensure that the design addresses all functional and non-functional requirements.
2. **Correctness:** Check that the design logic and component responsibilities are appropriate.
3. **Performance:** Determine whether the proposed architecture can meet response-time and throughput requirements.
4. **Security:** Verify authentication, authorization, encryption, and protection of sensitive data.
5. **Scalability:** Check whether the architecture can support an increase in users, transactions, and data.
6. **Maintainability:** Ensure that components can be modified, tested, and maintained independently where appropriate.
7. **Reliability and availability:** Check for suitable fault handling, redundancy, backup, and recovery mechanisms.
8. **Interface compatibility:** Verify that components and external systems communicate correctly.

Design verification versus architecture verification

Design verification	Architecture verification
Focuses on detailed design elements.	Focuses on the overall system structure.
Checks classes, algorithms, modules, and interfaces.	Checks components, services, dependencies, and deployment structure.
Often examines detailed design documents and specifications.	Often examines architecture diagrams, system boundaries, and technology choices.
Example: checking a password-validation algorithm.	Example: checking the separation of authentication and payment services.

Techniques used

1. Design reviews Developers, architects, testers, and other stakeholders examine design documents to identify omissions, inconsistencies, and incorrect assumptions.
2. Walkthroughs The designer explains the design step by step, and reviewers ask questions or suggest improvements.
3. Inspections A structured review is conducted using defined roles, checklists, and documented findings.
4. Architecture analysis The architecture is evaluated against important quality attributes, including performance, security, availability, and scalability.

5. UML diagram verification Use-case, class, sequence, activity, and component diagrams are checked for consistency with requirements.
6. Interface verification The inputs, outputs, data formats, error responses, and communication protocols of interacting components are reviewed.
7. Prototyping and simulation A prototype or architectural proof of concept may be built to investigate uncertain technical decisions, such as whether a database can handle the expected transaction volume.

General procedure

1. Study the software requirements specification (SRS).
2. Identify functional and non-functional requirements.
3. Examine the architectural and detailed design documents.
4. Prepare verification checklists and review criteria.
5. Inspect components, interfaces, dependencies, and data flows.
6. Evaluate risks relating to security, performance, and reliability.
7. Record defects, omissions, and design inconsistencies.
8. Correct the identified problems.
9. Re-review the design and document the results.

Example

Consider an online banking application with the following components:

- User interface
- Authentication service
- Account management service
- Transaction processing service
- Database
- Notification service

During architecture verification, the team checks whether users must authenticate before initiating transactions, whether transaction processing prevents duplicate payments, whether sensitive data is protected, and whether failed transactions can be handled safely.

If the design allows the same payment request to be processed twice because it lacks duplicate-request protection, the team should identify and resolve this design risk before release.

Advantages

- Identifies design defects early.
- Reduces rework and development costs.
- Improves system security and maintainability.
- Helps prevent performance and scalability problems.
- Improves confidence that the design satisfies requirements.

Limitations

- Requires experienced reviewers.
- Can be time-consuming for large systems.
- Some performance and reliability issues cannot be confirmed through document reviews alone.
- Incomplete or incorrect requirements may lead to misleading conclusions.

Deployment Testing

Definition

Deployment testing is the process of verifying that a software application can be installed, configured, upgraded, migrated, and operated correctly in its intended environment.

It checks not only the application itself but also its dependencies, configuration, infrastructure, permissions, and deployment procedures.

Deployment testing is commonly performed in a staging or pre-production environment before a production release. Important checks may also be performed immediately after production deployment.

Objectives

- Verify successful installation and startup.
- Ensure that configuration values are correct.
- Check compatibility with the operating system, database, and other dependencies.
- Verify that the application can connect to required services.
- Ensure that upgrades preserve existing data.
- Confirm that rollback and recovery procedures work.
- Detect environment-specific problems.
- Reduce the risk of deployment failures and service interruptions.

Types of deployment testing

1. **Installation testing** Checks whether the application installs correctly on a new system. Example: Installing a desktop application on a supported operating system.
2. **Configuration testing** Checks environment-specific settings such as database connection strings, API endpoints, access permissions, and environment variables.
3. **Upgrade testing** Verifies that a newer version can be installed without breaking existing functionality or losing data.
4. **Migration testing** Checks whether data is transferred correctly from an older database schema or system to a newer one.
5. **Rollback testing** Verifies that the system can return to a previous stable version if deployment fails.
6. **Compatibility testing** Checks whether the application works with supported operating systems, browsers, databases, hardware, and software dependencies.
7. **Smoke testing after deployment** Runs a small set of critical tests to establish that the deployed build is operational.
8. **Uninstallation testing** Checks whether the software can be removed correctly without leaving unwanted files, services, or configuration entries.

Deployment testing procedure

1. Review deployment instructions and environment requirements.
2. Prepare the test server or staging environment.
3. Verify dependencies, permissions, network connectivity, and configuration.
4. Install or deploy the application.
5. Check startup logs and service status.
6. Test critical features, such as login, data retrieval, and database operations.
7. Verify integrations with external services.
8. Test upgrade, migration, and rollback procedures where applicable.
9. Record failures and correct configuration or deployment defects.
10. Prepare a deployment test report and release recommendation.

Example

Suppose an e-commerce application works correctly on a developer's computer but fails after deployment to a cloud server.

Deployment testing may reveal that:

- The production database URL is incorrect.
- A required environment variable is missing.
- The application lacks permission to write uploaded files.
- A required software dependency is not installed.
- A network rule prevents communication with the payment gateway.

These are deployment or environment-related problems even though the application may have passed functional tests earlier.

Advantages

- Reduces production deployment failures.
- Improves environment consistency.
- Protects existing data during upgrades.
- Verifies operational readiness.
- Improves recovery and rollback preparedness.

Limitations

- Requires representative test environments.
- Some production-only issues may remain undetected.
- Infrastructure differences can make results difficult to reproduce.
- Testing deployment procedures can require additional time and resources.

Beta Testing

Definition

Beta testing is a type of software testing in which a nearly complete software product is made available to selected external users or a broader user community before its official release.

These users operate the software in real-world environments and provide feedback about defects, usability, compatibility, performance, and overall user satisfaction.

Beta testing is generally conducted after internal testing, such as system testing and alpha testing, has reached an acceptable level.

Objectives of beta testing

1. Identify defects that were not detected during internal testing.
2. Evaluate the software under real-world conditions.
3. Collect feedback about usability and user experience.
4. Check compatibility with different devices, operating systems, and network conditions.
5. Evaluate whether the software meets user expectations.
6. Identify missing features or confusing workflows.
7. Reduce the risk of releasing a product with serious problems.

Types of beta testing

1. Closed beta testing The software is provided to a limited group of invited users.

- Participation is restricted.
- Feedback can be collected in a controlled manner.
- It is useful for products that require confidentiality or careful monitoring.

Example: A company invites 200 selected customers to test a new mobile banking application.

2. Open beta testing The software is made available to a wider audience, sometimes with few restrictions on participation.

- A larger number of users can participate.
- The product is tested across more varied environments.
- Feedback volume can be high and more difficult to manage.

Example: A software company releases a public beta version of its new browser.

Beta testing process

1. Complete the necessary internal testing.
2. Define the scope and objectives of the beta program.
3. Select suitable beta users.
4. Prepare the beta version and provide installation or access instructions.
5. Explain how users can report defects and feedback.
6. Allow users to operate the software in their normal environments.
7. Collect crash reports, usability feedback, and defect reports.
8. Analyze and prioritize the findings.
9. Correct important defects.
10. Retest the corrections and decide whether the product is ready for general release.

What is evaluated during beta testing?

- Functionality and ease of use.
- Compatibility with real devices and configurations.
- Performance under typical user workloads.
- Unexpected errors and crashes.
- Accessibility and user experience.
- Reliability during normal operation.
- Whether workflows match users' actual needs.

Example

A company develops a food-delivery application and releases a beta version to 1,000 users.

During testing, users report that:

- The application sometimes crashes during checkout.
- Restaurant locations are displayed incorrectly.
- The payment confirmation message is confusing.
- The application works poorly on slower internet connections.

The development team analyzes the reports, fixes the problems, and conducts regression testing before releasing the final version.

Advantages

- Provides feedback from actual users.
- Reveals environment-specific defects.
- Helps improve usability and product quality.
- Identifies problems that controlled laboratory testing may miss.
- Builds confidence before general release.

Limitations

- Feedback may be incomplete or biased.
- Some users may not report defects clearly.
- Beta results may not represent the entire target population.
- An unstable beta release can negatively affect user confidence.
- Sensitive data and privacy must be protected.

Alpha testing versus beta testing

Alpha testing	Beta testing
Usually takes place before beta testing.	Usually follows alpha testing.
Generally conducted internally or in a controlled environment.	Conducted by external users in real-world environments.
Developers and internal testers commonly participate.	Selected customers or public participants commonly participate.
Defects can be investigated under controlled conditions.	Defects may depend on varied devices, networks, and user behavior.

Scalability Testing

Definition

Scalability testing is a type of performance testing that determines how well a software system handles increasing workloads and whether its capacity can grow while meeting defined performance requirements.

A scalable system should be able to support more users, transactions, requests, or data by using additional resources or improving resource utilization.

Scalability testing is particularly important for web applications, cloud platforms, e-commerce systems, banking applications, and distributed systems.

Objectives

- Determine how performance changes as workload increases.
- Measure the maximum workload that meets performance requirements.
- Identify bottlenecks in CPU, memory, databases, networks, and application services.
- Evaluate the effectiveness of adding hardware or service instances.
- Estimate future infrastructure requirements.
- Check whether performance remains acceptable during growth.

Types of scalability

1. Vertical scalability (scaling up)

Increasing the resources available to a single server or machine.

Examples:

- Increasing RAM.
- Using a more powerful CPU.
- Moving to a database server with greater processing capacity.

Advantages include simpler infrastructure in some cases, but the machine has physical or platform limits.

2. Horizontal scalability (scaling out)

Adding more servers or application instances to distribute the workload.

Examples:

- Adding application servers behind a load balancer.
- Increasing the number of container replicas.
- Distributing database read operations across replicas where appropriate.

Horizontal scaling can improve capacity and fault tolerance, but it introduces coordination, networking, and data-consistency challenges.

Important scalability metrics

Metric	Meaning
Response time	Time taken to respond to a request.
Throughput	Number of requests or transactions processed per unit of time.
Concurrent users	Number of users active at the same time.
CPU utilization	Percentage of CPU processing capacity in use.
Memory utilization	Amount of RAM used by the application and supporting services.
Error rate	Proportion of requests that fail.
Resource cost	Infrastructure cost required to support the workload.

Scalability testing procedure

1. Define the workload and performance requirements.
2. Establish a baseline under normal operating conditions.
3. Prepare realistic test data and a representative test environment.
4. Gradually increase the workload.
5. Monitor response time, throughput, error rate, CPU, memory, and database performance.
6. Identify the point at which performance no longer meets requirements.
7. Add resources or optimize the system.
8. Repeat the tests to evaluate the effectiveness of scaling.
9. Compare the results against performance targets.
10. Document capacity limits and recommendations.

Example

Suppose an online shopping website must support 10,000 concurrent users with a 95th-percentile response time below 2 seconds.

The test team gradually increases the number of concurrent users.

Concurrent users	P95 response time	Error rate
1,000	0.5 seconds	0.1%
3,000	0.8 seconds	0.2%
6,000	1.3 seconds	0.4%
10,000	1.9 seconds	0.5%
12,000	3.4 seconds	2.5%

These figures are illustrative. The results suggest that the application meets the stated response-time target at 10,000 users but struggles at 12,000. Further testing is needed to identify the bottleneck and determine whether additional resources can improve capacity.

Common bottlenecks

- Insufficient CPU or memory.
- Slow database queries.
- Database connection limits.
- Network congestion.
- Inefficient algorithms.
- Lock contention and shared-resource contention.
- Poor load balancing.
- External services with limited capacity.

Advantages

- Helps estimate future capacity.
- Identifies performance bottlenecks.
- Supports infrastructure planning.
- Improves user experience during growth.
- Helps control resource and cloud infrastructure costs.

Limitations

- Large-scale testing can be expensive.
- Test environments may not reproduce production behavior exactly.
- Realistic workload models are difficult to create.
- Some bottlenecks appear only at very high concurrency levels.

Reliability Testing

Definition

Reliability testing evaluates whether software performs its required functions consistently without failure for a specified period under specified conditions.

Reliability is especially important in systems where failures can cause major financial loss, safety risks, service interruption, or data corruption.

Examples include banking systems, hospital information systems, aircraft software, telecommunications systems, and industrial control applications.

Objectives

1. Measure how frequently failures occur.
2. Verify that the application operates consistently over time.
3. Detect memory leaks and resource exhaustion.
4. Identify intermittent and difficult-to-reproduce defects.
5. Evaluate recovery after failures.
6. Determine whether reliability requirements are satisfied.

Types of reliability testing

1. Load-based reliability testing Runs the application under an expected workload for an extended period to detect failures during normal use.

2. Endurance or soak testing Operates the system continuously for an extended duration to detect memory leaks, resource exhaustion, and gradual performance degradation.

3. Recovery-related reliability testing Introduces selected failures and checks whether the application recovers correctly without unacceptable data loss.

4. Fault-injection testing Deliberately introduces failures, such as network interruptions or unavailable services, to evaluate the system's response.

5. Long-duration stability testing

Checks whether the application remains stable over hours, days, or longer periods as appropriate to the requirements.

Reliability metrics

1. Mean Time Between Failures (MTBF) MTBF is the average operating time between failures of a repairable system.

Formula:

MTBF = Total Operating Time / Number of Failures

Example:

If a system operates for 1,000 hours and experiences 5 failures:

MTBF = 1,000 / 5 = 200 hours.

Therefore, the average operating time between failures is 200 hours.

2. Mean Time To Repair (MTTR) MTTR is the average time required to repair and restore a failed system.

Formula:

MTTR = Total Repair Time / Number of Repairs

Example:

If five failures require a total of 10 hours to repair:

MTTR = 10 / 5 = 2 hours.

3. Availability Availability measures the proportion of time a system is operational.

Formula:

Availability = MTBF / (MTBF + MTTR) × 100%

Example:

MTBF = 200 hours and MTTR = 2 hours.

Availability = 200 / (200 + 2) × 100%

Availability ≈ 99.01%.

4. Failure rate

Failure rate measures the frequency of failures relative to operating time.

Formula:

Failure Rate = Number of Failures / Total Operating Time

Example:

For 5 failures in 1,000 operating hours:

Failure Rate = 5 / 1,000 = 0.005 failures per hour.

Reliability testing procedure

1. Define reliability targets and operating conditions.
2. Prepare the test environment and realistic workloads.
3. Run the application for the required duration.
4. Monitor crashes, errors, memory usage, CPU, and response time.
5. Record each failure and its circumstances.
6. Investigate root causes.
7. Apply fixes and repeat the relevant tests.
8. Verify recovery and data integrity where appropriate.
9. Calculate the required reliability metrics.
10. Prepare a reliability test report.

Example

1. A hospital management system is tested continuously for 72 hours while processing simulated patient registrations, appointments, and billing transactions.

During testing, the team discovers that memory usage increases steadily after each report-generation operation. After 48 hours, the application becomes unresponsive.

Investigation reveals a memory leak. Developers correct the defect and repeat the endurance test to verify that memory consumption remains stable.

2.Consider an online railway reservation system. The testing team operates the system continuously for 100 hours while simulating ticket bookings, cancellations, payments, and enquiries.

During testing:

- The application crashes twice.
- One memory leak is identified.
- Some requests fail when the database connection is interrupted.
- The system recovers after a restart.

The development team corrects the memory leak, improves database error handling, and repeats the tests.

The system demonstrates improved reliability when it completes operations consistently, handles failures appropriately, and meets the defined reliability targets.

Advantages

- Improves system stability.
- Reduces unexpected downtime.
- Helps detect defects that emerge over time.
- Improves confidence in critical applications.
- Supports availability and maintenance planning.

Limitations

- Long-duration tests can require significant resources.
- Rare failures may not occur during the test period.
- Laboratory conditions may differ from actual operational conditions.
- Reliability estimates are only as useful as the assumptions and data used.

Stress Testing

Definition

Stress testing is a type of performance testing that evaluates how a software system behaves when subjected to workloads or resource constraints beyond its normal operating limits.

Its primary purpose is to determine how the system behaves under extreme conditions, identify its breaking point, and verify whether it fails safely and recovers correctly.

Stress testing is particularly important for systems that experience sudden traffic spikes, heavy transaction volumes, limited resources, or unpredictable demand.

Objectives of stress testing

- Determine the maximum workload a system can tolerate.
- Identify performance bottlenecks and resource limitations.
- Observe how the system behaves when overloaded.
- Check whether errors are handled gracefully.
- Verify that data is not corrupted during overload.
- Evaluate recovery after the excessive workload is removed.
- Determine whether the system meets its overload-handling requirements.

Types of stress testing

1. Application stress testing Tests the application under extreme numbers of requests, transactions, or concurrent users.

Example: Sending a very large number of requests to an online shopping website.

2. CPU stress testing Creates heavy computational workloads to evaluate the effect of high CPU utilization.

3. Memory stress testing Consumes large amounts of memory to evaluate how the system behaves under memory pressure.

4. Network stress testing Introduces high network traffic, latency, packet loss, or connection interruptions to assess system behavior.

5. Database stress testing Generates high volumes of database queries and transactions to identify locking, connection, and query-performance problems.

6. Spike testing Suddenly increases the workload to evaluate how the system responds to abrupt demand changes.

Example: Traffic increases from 1,000 to 15,000 concurrent users within a short period.

7. Recovery testing under stress Checks whether the system returns to normal operation after the excessive workload has been removed.

Stress testing procedure

1. Define the normal operating workload and expected limits.
2. Establish performance baselines.
3. Prepare a representative test environment.
4. Gradually increase workload or resource pressure.
5. Monitor response time, throughput, error rate, and resource usage.
6. Identify the point at which the system stops meeting its requirements.
7. Observe whether the system rejects requests safely or becomes unavailable.
8. Reduce the workload and observe recovery.
9. Check data integrity and the completion status of transactions.
10. Document bottlenecks, failure modes, and recommendations.

Example

Suppose a ticket-booking application normally supports 5,000 concurrent users.

The test team gradually increases the load beyond the expected capacity.

Concurrent users	Average response time	Observed behavior
2,000	0.4 seconds	Normal operation
5,000	0.9 seconds	Acceptable performance
8,000	2.1 seconds	Performance degrades
12,000	6.5 seconds	Many requests time out
15,000	Not meaningful due to failures	Service becomes unavailable

These are illustrative figures. The results indicate that the system struggles under extreme load. The team should investigate the bottleneck and verify whether the system can recover without corrupting booking or payment data.

Stress testing versus load testing

Load testing	Stress testing
Evaluates behavior under expected workloads.	Evaluates behavior under extreme or excessive workloads.
Checks whether performance targets are met.	Checks limits, failure behavior, and recovery.
Usually remains within the intended operating range.	Deliberately pushes beyond that range.
Example: Testing 5,000 expected concurrent users.	Example: Increasing workload to 15,000 users to observe failure behavior.

Advantages

- Reveals capacity limits and bottlenecks.
- Helps prevent overload-related outages.
- Identifies unsafe failure behavior.
- Evaluates recovery mechanisms.
- Helps teams plan capacity and improve resilience.

Limitations

- May consume substantial infrastructure resources.

- Can disrupt shared test environments.
- Results may not accurately predict every real-world failure.
- Requires careful control to prevent unintended effects on production systems.

Important: Stress testing should normally be conducted in a controlled test environment. Production stress testing requires explicit authorization, safeguards, and an appropriate operational plan.

Acceptance Testing

Definition

Acceptance testing is the process of evaluating a software system to determine whether it satisfies the agreed business requirements and is suitable for delivery or use.

It is generally performed after the system has passed the necessary earlier testing stages. Customers, end users, business representatives, or other authorized stakeholders may participate.

Acceptance testing focuses on whether the system meets its intended purpose rather than simply whether individual components work correctly.

It is commonly associated with the question:
"Are we building the right product?"

Objectives of acceptance testing

1. Verify that business requirements are satisfied.
2. Validate complete user workflows.
3. Confirm that the software meets the agreed acceptance criteria.
4. Identify missing or incorrectly implemented business functionality.
5. Verify readiness for operational use.
6. Check contractual or regulatory requirements when applicable.
7. Provide evidence for the final acceptance decision.

Types of acceptance testing
1. User Acceptance Testing (UAT)

End users or business representatives verify that the system supports real business activities.

Example: Bank employees verify that an application correctly handles deposits, withdrawals, transfers, and account statements.
2. Operational Acceptance Testing (OAT)

Checks whether the software is ready to be operated and maintained.

It may include:

- Backup and restoration.
- Disaster recovery.
- Monitoring and alerting.
- Security and access controls.
- Maintenance and operational procedures.
- Deployment and rollback readiness.

3. Contractual acceptance testing

Checks whether the software satisfies requirements agreed upon in a contract.
4. Regulatory acceptance testing

Verifies applicable regulatory, legal, or industry-specific requirements.
5. Alpha testing

Internal or controlled testing performed before wider external testing or release.

6. Beta testing

External users evaluate a nearly complete product in real-world conditions.

Alpha and beta testing are often part of broader release-readiness activities; their precise relationship to formal acceptance testing depends on the organization's process.

Acceptance testing process

1. Review business requirements and specifications.
2. Define measurable acceptance criteria.
3. Prepare an acceptance test plan.
4. Identify users, responsibilities, and test scenarios.
5. Select test cases that cover the acceptance criteria.
6. Prepare test data and a suitable test environment.
7. Execute the test cases.
8. Compare actual results with expected results.
9. Record defects and retest corrections.
10. Review coverage, outstanding defects, and risks.
11. Prepare the acceptance test report.
12. Obtain the authorized stakeholder's acceptance or rejection decision.

Acceptance Criteria

Definition

Acceptance criteria are the specific conditions that a software feature, component, or complete system must satisfy to be considered acceptable.

They define the expected behavior and provide a basis for determining whether testing has succeeded.

Acceptance criteria should be:

- **Clear:** Written so that stakeholders understand them.
- **Testable:** Capable of being verified.
- **Measurable where applicable:** Supported by objective thresholds.
- **Relevant:** Related to actual business needs.
- **Agreed upon:** Reviewed by the appropriate stakeholders.
- **Unambiguous:** Avoiding vague expressions such as “works well.”

Types of acceptance criteria

1. Functional criteria

Define the required business functionality.

Example: A registered user can place an order using a valid delivery address.

2. Performance criteria

Define response time, throughput, or other performance targets.

Example: The checkout request must complete within 2 seconds at the specified workload and percentile.

3. Security criteria

Define requirements for access control, authentication, and protection of sensitive information.

Example: An unauthenticated user cannot access account details.

4. Usability criteria

Define the expected ease of use or accessibility of a feature.

Example: A user receives a clear validation message when a required field is empty.

5. Data integrity criteria

Define requirements for data correctness and consistency.

Example: The order total stored in the database matches the amount confirmed to the customer.

Example: Acceptance criteria for an online shopping system

Consider the checkout feature of an e-commerce application.

- AC01: A user can add available products to the shopping cart.
- AC02: The system calculates the correct subtotal, tax, shipping charge, and final total.
- AC03: The user cannot place an order without a valid delivery address.
- AC04: A successful payment results in a confirmed order.
- AC05: A failed payment must not result in a falsely confirmed order.
- AC06: The customer receives an order confirmation after successful completion.
- AC07: The system prevents duplicate order creation when the same request is retried, according to the specified duplicate-prevention rules.

These criteria can be verified using appropriate test cases.

Test Case Selection

Definition

Test case selection is the process of identifying and choosing test cases that adequately verify the acceptance criteria and business requirements of a system.

The aim is to achieve meaningful coverage while using testing time and resources efficiently.

Factors affecting test case selection

1. Requirement coverage

Every acceptance criterion should be covered by one or more suitable test cases.

2. Business importance

Features that are essential to the business should receive appropriate testing priority.

3. Risk and impact

Features with serious consequences if they fail should be tested thoroughly.

4. Positive scenarios

Verify that valid inputs and normal user workflows work correctly.

5. Negative scenarios

Verify that invalid inputs, unauthorized requests, and failure conditions are handled correctly.

6. Boundary conditions

Test values at and around important limits, such as minimum quantities, maximum amounts, and field-length restrictions.

7. Integration scenarios

Verify that interacting services and external systems work together correctly.

8. Regression requirements

Include previously tested functionality that might have been affected by recent changes.

Techniques for selecting test cases

- **Equivalence partitioning:** Divide input data into groups expected to behave similarly, then select representative values.
- **Boundary-value analysis:** Test values at, just below, and just above important limits.
- **Decision-table testing:** Cover combinations of conditions and their corresponding outcomes.
- **State-transition testing:** Check behavior as the system moves between states.
- **Risk-based testing:** Prioritize cases according to the probability and impact of failure.
- **Requirements traceability:** Map test cases to individual requirements and acceptance criteria.

Example: Quantity field in an online order

Suppose the allowed order quantity is from 1 to 100 units.

Suitable test values include:

Test input	Purpose	Expected result
0	Below minimum	Rejected
1	Minimum valid value	Accepted
50	Typical valid value	Accepted
100	Maximum valid value	Accepted
101	Above maximum	Rejected
-1	Invalid negative value	Rejected
Text such as abc	Invalid data type	Rejected

This example demonstrates how boundary-value analysis and invalid-input testing can help select meaningful test cases.

Test Case Execution

Definition

Test case execution is the process of running selected test cases against the software, recording actual outcomes, comparing them with expected outcomes, and reporting failures.

Components of a test case

A test case commonly contains:

1. **Test case ID:** A unique identifier.
2. **Test objective:** What is being verified.
3. **Preconditions:** Conditions that must be satisfied before execution.
4. **Test data:** Inputs and data required.
5. **Execution steps:** Actions to perform.
6. **Expected result:** The required outcome.
7. **Actual result:** The observed outcome.
8. **Status:** Pass, Fail, Blocked, or Not Run.
9. **Defect reference:** An identifier linking the test to a reported defect, if applicable.

Steps for test case execution
Step 1: Prepare the test environment

Ensure that the required software build, database, network connections, test accounts, and configuration are available.
Step 2: Review the test cases

Confirm that the selected cases correspond to the acceptance criteria and have clear expected results.

Step 3: Prepare test data

Create appropriate valid, invalid, boundary, and exceptional data.

Step 4: Execute the test steps

Follow the documented instructions and perform the required actions.

Step 5: Record actual results

Capture the observed behavior, including error messages, output values, logs, and relevant evidence.

Step 6: Compare expected and actual results

Determine whether the observed behavior matches the defined acceptance criteria.

Step 7: Assign a test status

- **Pass:** Actual behavior matches the expected result.
- **Fail:** Actual behavior differs from the expected result.
- **Blocked:** Execution cannot proceed because of a dependency or environment problem.
- **Not Run:** The test has not been executed.

Step 8: Report defects

Document failures with reproduction steps, severity, relevant evidence, and expected versus actual behavior.

Step 9: Retest corrections

After a defect is fixed, rerun the affected test case.

Step 10: Perform regression testing

Check that the correction has not introduced problems in other parts of the application.

Step 11: Evaluate acceptance readiness

Review the results, requirement coverage, unresolved defects, and stakeholder-defined exit criteria.

Worked Example: Acceptance test case execution

Consider the login feature of a banking application.

Acceptance criterion: A registered user with valid credentials must be able to log in, while an incorrect password must not grant access.

Test case TC-LOGIN-01

Objective

Verify successful login

Preconditions

- The user account exists and is active.
- The application is available.
- The correct test credentials are known.

Test steps

1. Open the login page.
2. Enter the registered username.
3. Enter the correct password.
4. Select the Login button.

Expected result

The user is authenticated and redirected to the authorized account dashboard.

Illustrative actual result

The dashboard opens and displays the correct account information.

Execution status

Pass

Test case TC-LOGIN-02
Objective
Verify incorrect password handling
Preconditions
The user account exists and is active.
Test steps

1. Open the login page.
2. Enter the registered username.
3. Enter an incorrect password.
4. Select the Login button.

Expected result
Access is denied and an appropriate error message is displayed.
Illustrative actual result
The application opens the dashboard despite the incorrect password.
Execution status

Fail

Required action: Report the authentication defect, correct it, and repeat the failed test. Perform relevant regression testing before accepting the login feature.
The actual results above are illustrative examples, not results from a real application.

Acceptance Test Report

An acceptance test report summarizes the testing performed and provides evidence for the acceptance decision.

A typical report contains:

- Project and software version.
- Testing scope and objectives.
- Test environment.
- Acceptance criteria.
- Number of test cases executed.
- Number of passed, failed, blocked, and unexecuted tests.
- Defects identified and their severity.
- Requirement coverage.
- Outstanding risks and limitations.
- Final recommendation and approval.

For example, a report might state that 95 out of 100 test cases passed, 3 failed, and 2 were blocked. These figures alone do not establish acceptance: the team must also evaluate critical failures, required coverage, and the agreed exit criteria.

Advantages of acceptance testing

- Confirms alignment with business requirements.
- Helps prevent unsuitable software from being released.
- Identifies missing or incorrect functionality.
- Improves stakeholder confidence.
- Provides documented evidence for release decisions.

Limitations of acceptance testing

- Depends on the quality of requirements and acceptance criteria.
- Can be time-consuming when many workflows must be tested.

- Inadequate test data may produce misleading results.
- Stakeholders may disagree about whether a feature is acceptable.
- Passing acceptance tests does not guarantee that all defects have been discovered.

Comparison of the Seven Testing Types

Testing type	Main question	Primary focus
Design/Architecture verification	Is the design consistent with requirements and constraints?	Design correctness and quality attributes
Deployment testing	Can the software be installed and operated correctly?	Installation, configuration, upgrades, and environment
Beta testing	How does the software behave for real users?	Real-world experience and external feedback
Scalability testing	Can the system handle increasing workloads?	Capacity growth and performance
Reliability testing	Does the system operate consistently over time?	Failure frequency and stability
Stress testing	How does the system behave under extreme conditions?	Limits, failure behavior, and recovery
Acceptance testing	Does the software satisfy the agreed requirements?	Business suitability and acceptance decision